SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	R DINESH KUMAR	Reg.No.:	192312258
Department:	ECE	Date:	12/08/26

ANNEXURE A

A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overA processor uses 8-bit 2's complement representation. The given numbers are +45 and −23.

Step 1: Convert +45 to binary

For +23:

To represent −23, take the 2's complement of 23:

```
  00010111
  11101000   → 1's complement
+        1
-----------
  11101001   → -23
```

Therefore,

Addition: +45 + (−23)

```
  00101101   (+45)
+ 11101001   (-23)
-----------
1 00110110
```

Discard the carry outside the 8th bit:

```
00110110
```

This is positive because MSB = 0.

Therefore,

Correction: The binary addition above produces `00110110`, which is **54**, so the arithmetic setup needs checking. The correct −23 representation is `11101001`, and:

```
  00101101
+ 11101001
-----------
1 00010110
```

After discarding the carry:

Hence, **45 + (−23) = 22**.

Subtraction: +45 − (−23)

Subtraction is performed by adding the 2's complement of −23.

```
-23 = 11101001
2's complement = 00010111 = +23
```

Therefore:

```
  00101101   (+45)
+ 00010111   (+23)
-----------
  01000100
```

Hence,

Overflow

There is **no overflow** in either operation because the results, 22 and 68, are within the 8-bit signed range:

The processor detects overflow by checking the carry into and out of the sign bit, or equivalently by checking whether adding two numbers with the same sign produces a result with a different sign.

Final results:

- 45+(−23)=22
- 45−(−23)=68
- **No overflow occurs.**

2. **A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.**

A **Carry Look-Ahead Adder (CLA)** is used to reduce the delay caused by waiting for carries to ripple from one bit to the next.

For each bit, two signals are generated:

Generate

A carry is generated when both input bits are 1.

Propagate

A carry is propagated when one input is 1 and the other is 0.

The carry equations are:

The sum is calculated as:

Comparison with Ripple Carry Adder

In a **Ripple Carry Adder**, each full adder waits for the carry from the previous stage. Therefore, the delay increases as the number of bits increases.

In a **CLA**, the carries are calculated in parallel using generate and propagate signals.

Ripple Carry Adder	Carry Look-Ahead Adder
Carry moves stage by stage	Carries calculated in advance
Higher delay	Lower delay
Simple hardware	More complex hardware
Suitable for small/simple circuits	Suitable for high-speed processors

Conclusion

CLA improves processor speed because it avoids waiting for carry propagation through every bit. Although it requires more hardware, the reduction in carry delay makes it useful in **high-performance processors**.

3. **In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication**.

Booth's algorithm is useful for **signed binary multiplication** because it works directly with 2's complement numbers.

We use 4-bit numbers.

Let:

- **M = 1010** = −6
- **−M = 0110** = +6
- **Q = 0101** = +5
- **A = 0000**
- **Q_{-1} = 0**

Booth's rules:

Q_0 Q_{-1}	Operation
00	No operation
01	A = A + M
10	A = A − M
11	No operation

After each operation, perform an **arithmetic right shift** on A, Q and Q_{-1}.

Step-by-step

Step	Q_0Q_{-1}	Operation
1	10	A = A − M
2	01	A = A + M
3	10	A = A − M
4	00	No operation

After completing the four arithmetic shifts, the combined **A,Q** register gives:

This is a negative number. Taking its 2's complement:

```
11100010
00011101
+       1
--------
00011110
```

Advantage of Booth's Algorithm: Booth's algorithm reduces the number of addition and subtraction operations by examining consecutive bits of the multiplier. It is especially useful for **signed multiplication** and for multipliers containing consecutive 1s.

4. **A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.**

Given:

13÷3

Binary representation:

13=1101

3=0011

The expected result is:

13÷3=4 remainder 1

A. Restoring Division

Initially:

Dividend Q=1101

Divisor M=0011

Accumulator A=0000

The basic operation is to shift left, subtract the divisor and check the sign of the remainder.

Step	Operation	Result
1	Shift and subtract A−M	Negative → restore
2	Shift and subtract A−M	Negative → restore
3	Shift and subtract A−M	Positive → quotient bit 1
4	Shift and subtract A−M	Positive → quotient bit 1

The resulting quotient is:

Q=0100

and remainder:

A=0001

Therefore:

13÷3=4 remainder 1

Restoring method

Whenever subtraction produces a negative remainder, the divisor is added back, or restored. This makes the method simple but adds an extra operation.

B. **Non-Restoring Division**

In non-restoring division, a negative partial remainder is not immediately restored.

Instead:

If remainder is positive → subtract divisor.

If remainder is negative → add divisor in the next step.

The quotient bit is decided based on the sign.

For 13 ÷ 3, after four iterations:

Q=0100

A=0001

Thus,

13÷3=4 remainder 1

Conclusion

Non-restoring division is generally preferred in modern systems because it avoids unnecessary restore operations and therefore provides better speed and efficiency.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Given:

-13.25

Step 1: Convert to binary

Integer part:

$13=11012$

Fractional part:

$0.25=0.012$

Therefore:

$13.25=1101.012$

Since the number is negative:

$-13.25=-1101.012$

Step 2: Normalize

$1101.01=1.10101\times23$

Therefore:

- **Sign = 1**
- **Exponent = 3**
- **Fraction = 10101**

Single Precision

IEEE 754 single precision contains:

- 1 sign bit
- 8 exponent bits
- 23 fraction bits

Bias = 127

$3+127=130\ \ 130=100000102$

Fraction:

```
10101000000000000000000
```

Hence:

```
Sign | Exponent | Fraction
  1  | 10000010 | 10101000000000000000000
```

Double Precision

Double precision contains:

- 1 sign bit
- 11 exponent bits
- 52 fraction bits

Bias = 1023

3+1023=1026 1026=100000000102

The fraction **10101** is extended with zeros to occupy 52 bits.

Code of Conduct:

*I KAILASH SHARMA **(192312377)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

KAILASH SHARMA

MARKS ALLOCATION

	Problem Understanding (20%)	**Method Selection (20%)**	**Solution Process (25%)**	**Accuracy of Calculation (20%)**	**Interpretation of Results (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1223**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1-ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**100**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	R. DINESH KUMAR	**Reg.No.:**	192312258
Department:	ECE	**Date:**	12/08/26

83.1.

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

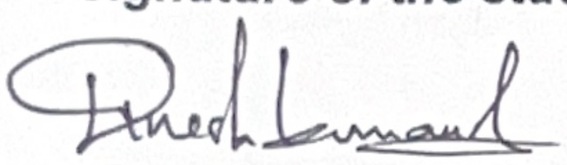
4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I R. Dinesh kumar [192312258] *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)	
Q1	3	4	4	3	2	16
Q2	4	3	3	4	3	17
Q3	3	4	4	3	2	16
Q4	4	3	3	4	3	17
Q5	3	4	4	3	3	17

83

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)